CSP תכנון ראשוני – בעיות
Constraint Satisfaction Problems

על פי שרה פולק

דגנית מוכתר-אבל

דגנית מוכתר-אבל

חומר תיאורטי

CPS בעיות

- סוג בעיות שבהן המטרה היא להקצות / להציב ערכים למשתנים תוך כדי שמירה שלא תהיה

- הפרה של אילוצים קיימים שמכתיבים אילו מבין הערכים מותר לשים במשתנים השונים לעיתים מחפשים את הפתרון האופטימלי ולא כל הקצאה של ערכים למשתנים

בעיות CPS

דוגמאות:

1. נניח שבמפעל מסוים רוצים לייצר שלושה מוצרים: לכל מוצר צריך כמות מסוימת של מים ושל חשמל כדי לייצר אותו. כמות המים והחשמל של המפעל מוגבלות. כמו כן לכל מוצר יש מחיר אחר בו אפשר למכור אותו. כמה מוצרים מכל סוג כדאי לייצר כדי למקסם רווחים? 2. הקצאת עובדים לביצוע משימות בצורה אופטימלית כך שהעלות הכוללת לביצוע המשימה תהיה מינימלית

3. התאמה של סטודנטים לאוניברסיטה, שיבוץ ילדים לגני ילדים בעירייה

4. תזמון מכונות לביצוע משימה גדולה שמורכבת מתתי משימות שלכל אחת זמן עבודה ויש סדר בין המשימות

5. חידת שמונה המלכות - שיבוץ 8 מלכות כך שבכל שורה ועמודה תהיה מלכה אחת ואף מלכה לא תאיים על האחרת לפי חוקי שח-מט

דגנית מוכתר-אבל

CSP אפיון בעיית

- הבעיה מוגדרת על ידי 3 רכיבים:
- **X** - סט של משתנים {X1, … , Xn}
- **D** - סט של תחומים, {D1, . . . , Dn}, אחד לכל משתנה (Domain) ○ מסט מורכב כל תחום
- **C** - Xi חוקי/מותר לכל {v1, . . . , vk} ערכים קבוצה של אילוצים המציינים שילובים מותרים מוגדר Ci כל אילוץ ○ (Constraint) של ערכים כזוג {scope, rel} כאשר:
  - ה- **scope** הוא רשימה של משתנים שמשתתפים באילוץ
  - **rel** הוא יחס המוגדר על ידי הערכים שהמשתנים יכולים לקבל

~~X - סט של משתנים {Xn , ... 1,X{~~

D - סט של תחומים, {Dn , . . . 1,D{ אחד לכל משתנה כל תחום

C - קבוצה של Xi חוקי/מותר לכל {vk , . . . 1,v} מורכב מסט ערכים

מוגדר Ci אילוצים המציינים שילובים מותרים של ערכים כל אילוץ

כזוג {rel ,scope} כאשר:

● scope -ה ● rel הוא רשימה של משתנים שמשתתפים באילוץ

יחס המוגדר על ידי הערכים שהמשתנים יכולים לקבל

אפיון בעיית CSP

- יחס יכול להיות מיוצג על ידי רשימה מפורשת של ערכים שמתאימים לאילוץ או כיחס מופשט
- דוגמה: אם יש אותו דומיין X 1,X למשתנים 2 {B,A אחר} ערך אילוץ שלכל אחד מהם חייב להיות התומך בשתי פעולות: בדיקה האם הרשומה היא איבר ביחס ומספור איברי היחס

אזי ניתן לכתוב את האילוץ בצורה מפורשת כך: $\langle (X_1, X_2), [(A, B), (B, A)] \rangle$

או בצורה מופשטת כך $\langle (X_1, X_2), X_1 \neq X_2 \rangle$:

דגנית מוכתר-אבל

coloring Map דוגמה בעיית

רוצים לצבוע את המדינות במפה כך שכל מדינה תצבע בצבע אחר משכנותיה

- למשל אוסטרליה המערבית וויקטוריה יכולות להיות כחולות
- אבל אוסטרליה המערבית ואוסטרליה המזרחית לא יכולות להיות באותו צבע

7

X - סט של משתנים {Xn , ... 1,X{

D - סט של תחומים, {Dn , . . . 1,D{, אחד לכל משתנה כל תחום

C - קבוצה של Xi חוקי/מותר לכל {vk , . . . 1,v} מורכב מסט ערכים

Ci מוגדר אילוצים המציינים שילובים מותרים של ערכים כל אילוץ

כזוג {rel ,scope {כאשר:

● ה- scope הוא רשימה של משתנים שמשתתפים באילוץ ● rel הוא

יחס המוגדר על ידי הערכים שהמשתנים יכולים לקבל

Map coloring דוגמה בעיית

רוצים לצבוע את המדינות במפה כך שכל מדינה תצבע בצבע אחר

משכנותיה

הגדרת הבעיה:

- X = {WA,NT, Q, NSW, V, SA, T}
- Di = {red, green, blue}
- C = {SA ≠ WA, SA ≠ NT, SA ≠ Q, SA ≠ NSW ,SA ≠ V, WA ≠ NT, NT ≠ Q, Q ≠ NSW ,NSW ≠ V}
- There are many possible solutions to this problem, such as ○ {WA = red,NT = green, Q = red,NSW = green, V =red,SA = blue, T = red }

Here we are using abbreviations; SA ≠ WA is a shortcut for <(SA,WA), SA ≠ WA>, where SA ≠ WA can be fully enumerated in turn as

{(red, green),(red, blue),(green, red),(green, blue),(blue, red),(blue, green)}

דגנית מוכתר-אבל

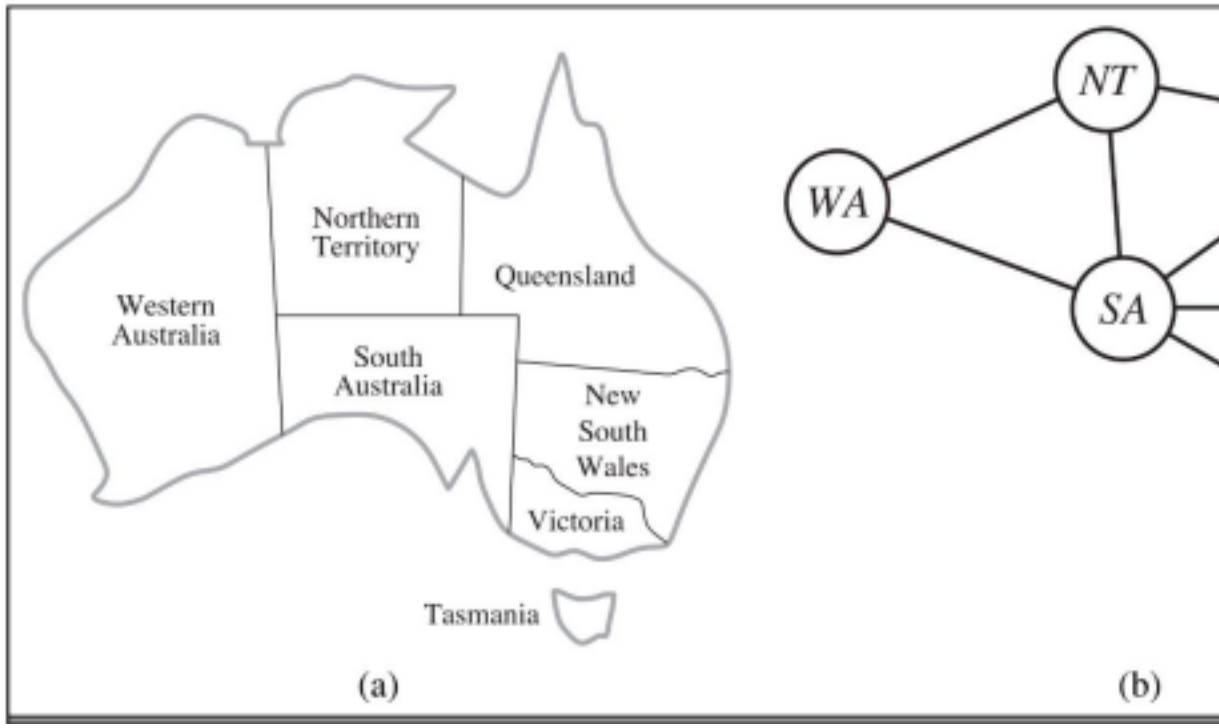
CSP פתרון בעיית

- כדי לפתור את הבעיה נצטרך להגדיר את מרחב המצבים של המשתנים והערכים שמתאימים להם כך שהם מקיימים את האילוצים
  - מוגדר על ידי השמת/הקצאת ערכים לחלק מהמשתנים או CSP-כל מצב ב לכולם {Xi = vi, Xj = vj, . . .}.

- **consistent** הקצאה שאינה מפרה אף אילוץ נקראת הקצאה עקבית
- **assignment complete** הקצאה מלאה היא כזו שבה כל משתנה מוקצה
- **assignment partial** - פתרון חלקי בו רק חלק מהערכים הוקצו
- פתרון לבעיית CSP הוא השמה מלאה ועקבית למשתנים

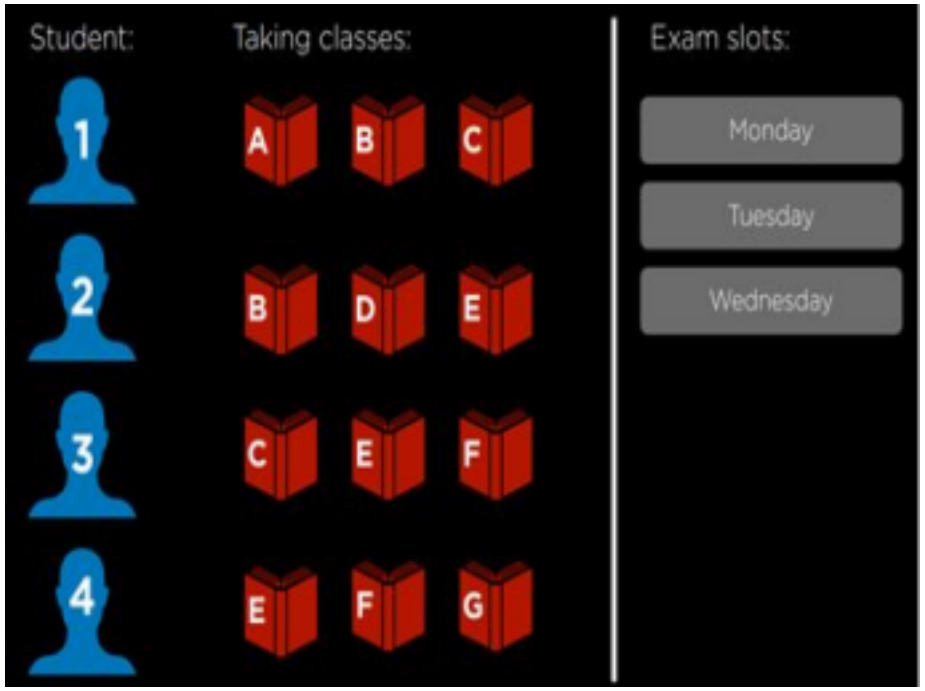
כגרף אילוצים שבו CSP נוח לייצג בעיית:

- הצמתים של הגרף הם המשתנים של הבעיה
- קשת מחברת בין שני משתנים שמשתתפים באילוץ



ייצוג בעיה כגרף

ייצוג בעיה כגרף - דוגמת לוח בחינות



- כל התלמידים 1 עד 4 לוקחים 3 קורסים

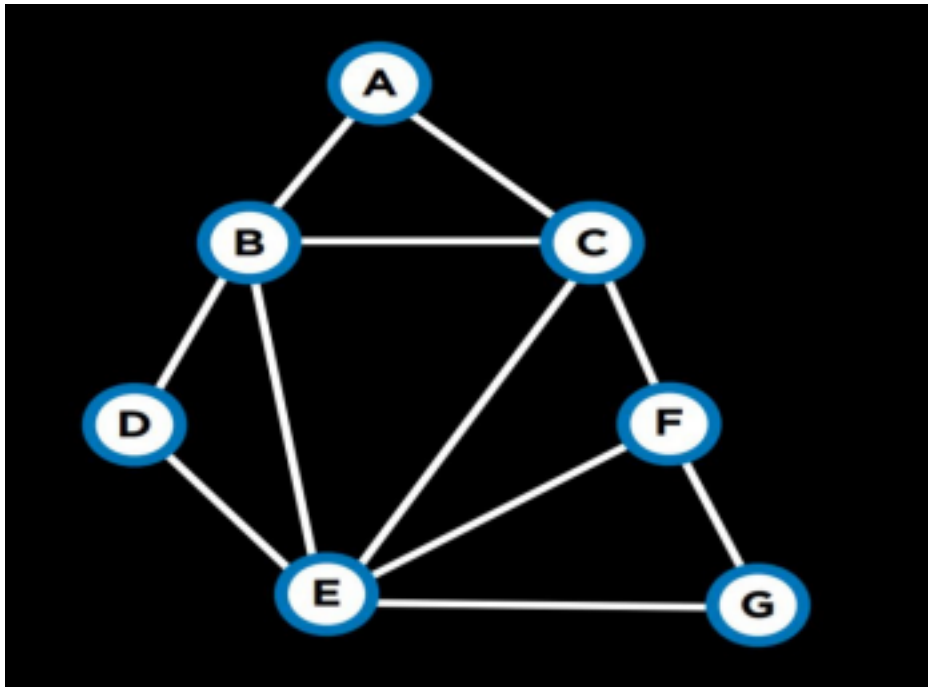
מרשימת 7 הקורסים האפשריים {G,..B,A}

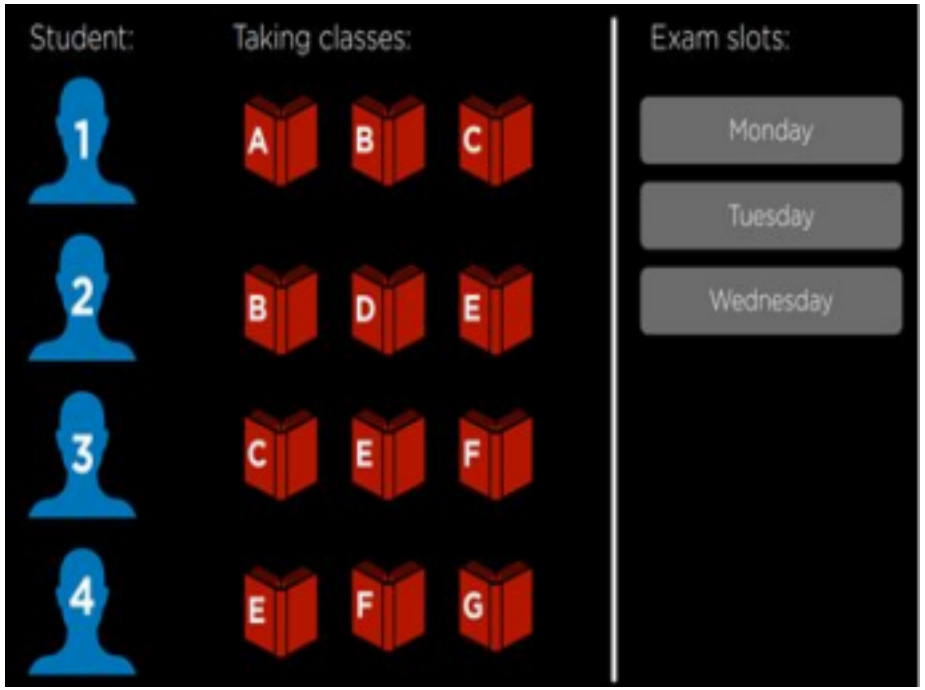
- בכל קורס יש מבחן והמבחנים יכולים להיות

Monday, Tuesday, Wednesday בימים

- לא יכולים להיות לתלמיד שני מבחנים באותו יום

ייצוג בעיה כגרף - דוגמת לוח בחינות





- המשתנים הם הקורסים
- הדומיין הם ימי הבחינה
- אילוץ אסור שתי בחינות באותו יום לאותו סטודנט 14
- (כל צומת היא קורס (משתנה
- קשת בין שני קורסים מתארת אילוץ שיש לקיים:
  לא ניתן לקבוע אותם באותו היום

CSP אפיון בעיית

:מושגים חשובים

- **Constraint Hard** - אילוץ שחייב להתקיים בפתרון
- **Constraint Soft** - אילוץ שרצוי לקיים בפתרון
- **Constraint Unary** - לא תהיה בחינה A אילוץ שמתייחס למשתנה בודד. בדוגמה בקורס {A ≠ Monday} ביום שני
- **Constraint Binary** - אילוץ שמערב 2 משתנים. למשל - לא יכולים לקבל את אותו ערך {A ≠ B} ((מבחן באותו יום לשני קורסים

Node Consistency

עקביות הצומת היא כאשר כל הערכים בתחום של משתנה עומדים באילוצים **האונריים** של

המשת
נה

דוגמה:

- התחום של כל קורס הוא {שני, שלישי, רביעי}, והאילוצים הם .B-ו A ,ניקח שני קורסים {A ≠ Mon, B ≠ Tue, B ≠ Mon, A ≠ B}. לא ,כעת A ולא B בתחום שיש מכיוון ,עקביים שלהם ערכים שהם לא מסוגלים לקחת.
- תהיה לו עקביות של צומת ,A אם נסיר את יום שני מהתחום של
- נצטרך להסיר את שני ויום שלישי מהתחום שלו ,B-כדי להשיג עקביות של צומת ב
- **למעשה, הסרנו מהתחום של כל משתנה את הערכים הלא חוקיים שלו**

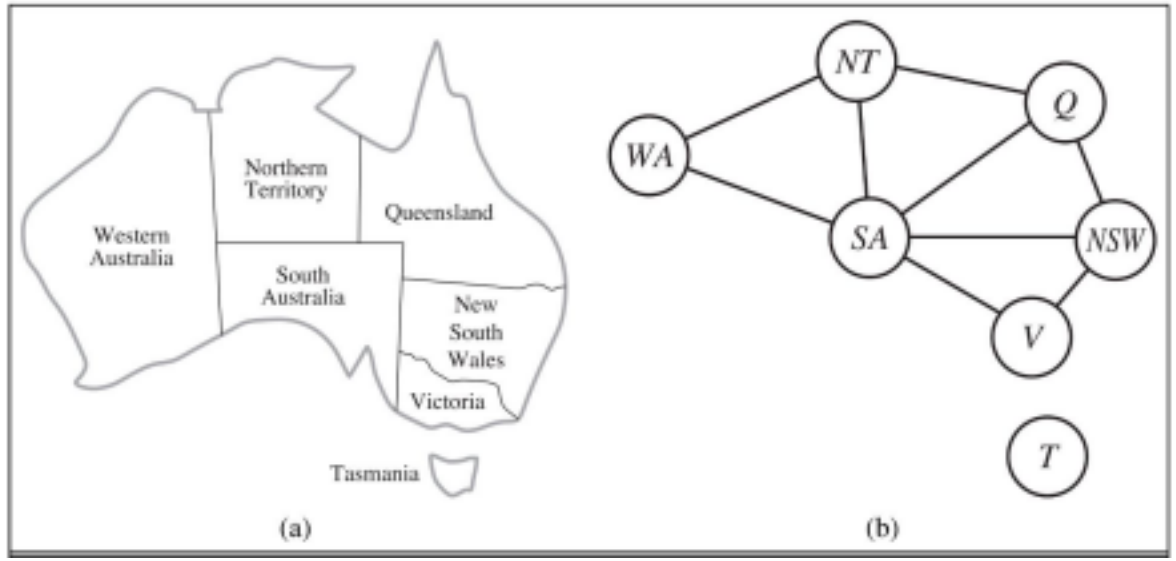
Node Consistency

עקביות הצומת היא כאשר כל הערכים בתחום של משתנה עומדים באילוצים **האונריים**

של
המשת
נה

דוגמה:

- אסור לצבוע SA למשל במפת הצבעים של מדינות באוסטרליה, נניח שאת {red,blue} במקום {blue ,green} בירוק ● אז אפשר להוריד את הצבע מהדומיין ולהשאיר {red,

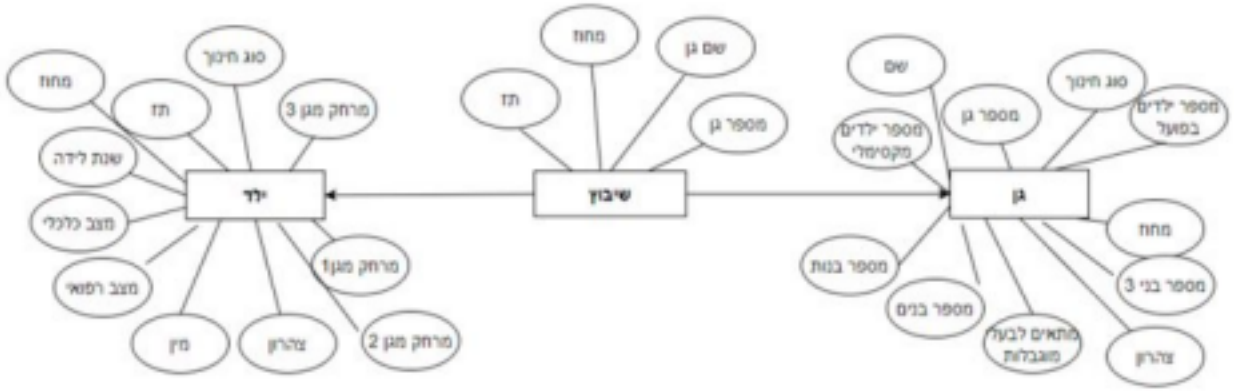


Arc Consistency

- עקביות קשת היא כאשר כל הערכים בתחום של משתנה עומדים באילוצים **הבינאריים** של המשתנה
- **נסיר אלמנטים מהדומיין של** Y, עם consistent-arc להיות X בייצוג גרפי, כדי להפוך את **תהיה בחירה אפשרית עבור X כדי שלכל בחירה עבור X Y**
- דוגמה : נניח הדומיין הבא לקורסים A ו- B והאילוץ הבינרי A ≠ B
- A:{Tuesday, Wednesday}
- B:{Wednesday}
- אם נסיר את Tuesday מהדומיין של A, יישאר רק הערך Wednesday
- אבל אם נסיר B עם consistent-arc הוא לא מצב A כי B, יגרום מצב שלא נוכל לתת ערך ל
  - יוכל לקבל את הערך B -ו Tuesday הערך A יוקצה ל A, מהדומיין של Wednesday את Wednesday

תכנון מחלקות לייצוג מאפייני הבעיה

- לאחר שהגדרתם את המשתנים, הדומיין והאילוצים תכננו את מסד נתונים והמחלקות שייטפלו במסד הנתונים
- למשל לשיבוץ ילדים בגן:
  - צריך טבלה לייצוג נתוני הגנים
    כל טבלה מגדירה משתנים, דומיי
  - אילוצים יוגדרו כחלק מהתוכנית
    JAVA בהתאם נגדיר מסד נתונינ



CSP

- תכנון ליניארי / algorithm -3AC
- גישוש נסוג_ עקיבה לאחור האלגוריתם ההונגרי
- בעיות אופטימיזציה – simplex

בשילוב גישוש נסוג algorithm -3AC

דגנית מוכתר-אבל

AC-3 algorithm

לכל [redacted] Consistency Arc

הקשתות

[redacted] בסרטון שמדגים את ריצת האלגוריתם [redacted]

- מבנה נתונים, למשל תור" = Agenda" בסרטון

דגנית מוכתר-אבל

function Revise(*csp*, *X*, *Y*):

- *revised* = *false*
- for *x* in *X.domain*:
    - if no *y* in *Y.domain* satisfies constraint for (*X*,*Y*):
        - delete *x* from *X.domain*
        - *revised* = true
- return *revised*

דגנית מוכתר-אבל

אלגוריתם Revise

הופך את X עם קשת עקבי להיות Y

- אלגוריתם זה מתחיל במעקב אם בוצע שינוי כלשהו בדומיין של X באמצעות המשתנה המתוקן
- יש ערך העונה על Y-ורואה אם ל X לאחר מכן, הקוד חוזר על כל ערך בדומיין של האילוצים
- אם כן, אל תעשה דבר, אם לא, הסר את הערך הזה מהדומיין של X

time complexity of $O(ed^3)$ and space complexity of $O(e)$

בהתחלה כל הקשתות מוכנסות לתור
כל קשת היא אילוץ שצריך לקיים

שליפת קשת מהתור ובדיקה אם היא קשת עקבית

ריק, זה אומר שבעיית שביעות רצון זו אינה ניתנת לפתרון X אם התחום המתקבל של
לקחת כל ערך בהתחשב Y-יכול לקחת שיאפשרו ל X-(מכיוון שאין ערכים ש
באילוצים)

אם נעשו שינויים כדי להפוך אותו לעקבי, יש צורך בפעולות נוספות. ● אם הבעיה לא
השתנה, עלינו לראות אם כל X נחשבה כבלתי פתירה בשלב הקודם, והתחום של
עדיין עקביות X-הקשתות הקשורות ל
ומוסיפים את הקשתות בינם Y מלבד X כלומר, אנחנו לוקחים את כל השכנים של ●
לתור כדי לבדוק אותם שוב X לבין

AC-3 algorithm

```
function AC-3( csp) returns false if an inconsistency is found and true otherwise
  inputs: csp, a binary CSP with components (X, D, C)
  local variables: queue, a queue of arcs, initially all the arcs in csp

  while queue is not empty do
    (Xi, Xj) ← REMOVE-FIRST(queue)
    if REVISE(csp, Xi, Xj) then
      if size of Di = 0 then return false
      for each Xk in Xi.NEIGHBORS - {Xj} do
        add (Xk, Xi) to queue
  return true
```

```
function REVISE( csp, Xi, Xj) returns true iff we revise the domain of Xi
  revised ← false
  for each x in Di do
    if no value y in Dj allows (x,y) to satisfy the constraint between Xi and Xj then
      delete x from Di
      revised ← true
  return revised
```

Figure 6.3 The arc-consistency algorithm AC-3. After applying AC-3, either every arc is arc-consistent, or some variable has an empty domain, indicating that the CSP cannot be solved. The name "AC-3" was used by the algorithm's inventor (Mackworth, 1977) because it's the third version developed in the paper.

AC-3
algorithm 25

ניתן לפתור את AC, לאחר שהבטחנו עקביות קשת לכל הקשתות בעזרת אלגוריתם 3

Backtracking Search

- גישוש נסוג / עקיבה לאחור הוא אלגוריתם חיפוש שלוקח בחשבון את המבנה של האילוצים

שצריכים להתקיים

- זו פונקציה רקורסיבית שמנסה להתאים ערכים למשתנים כל עוד מתקיימים האילוצים
- אם אילוץ מופר הוא נסוג אחורה ומנסה התאמה אחרת למשתנה

- function Backtrack(*assignment, csp*):
- if *assignment* complete:
  - return *assignment*
- *var* = Select-Unassigned-Var(*assignment, csp*)
- for *value* in Domain-Values(*var, assignment, csp*):
  - if *value* consistent with *assignment*:
    - add {*var* = *value*} to *assignment*
    - *result* = Backtrack(*assignment, csp*)
    - if *result* ≠ *failure*:
      - return *result*
    - *remove* {*var* = *value*} from *assignment*

דגנית מוכתר-אבל

Backtracking Search

להלן פסאודו קוד של פתרון רקורסיבי בעיית סיפוק

אילוצים:

פונקציה

ראשית:

אתחל את כל המשתנים לריקים 1.
בחר את אחד המשתנים והפעל את הפונקציה 2.

הרקורסיבית. פונקציה רקורסיבית

(מקבלת משתנה):

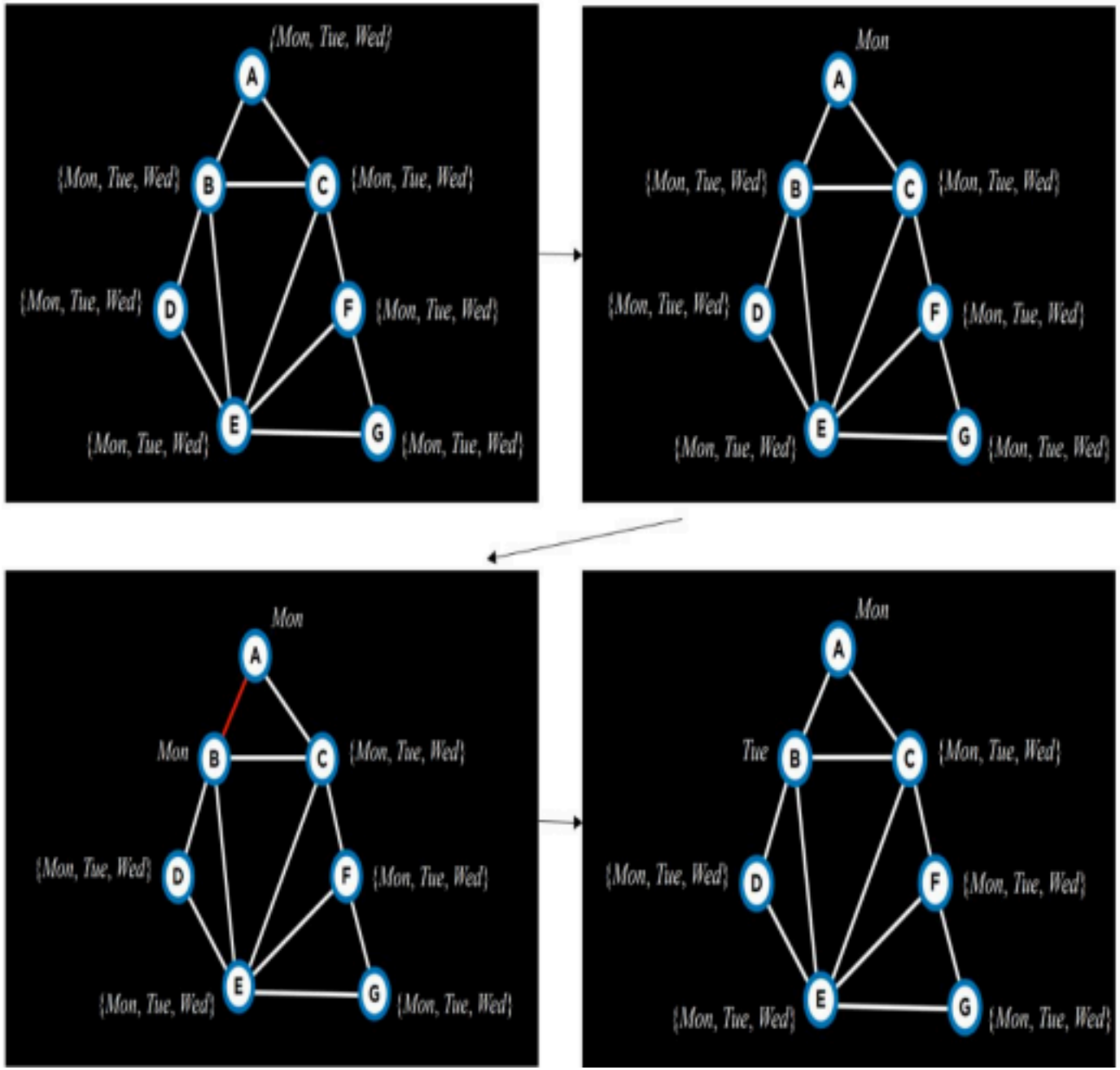
נסה להציב למשתנה את כל הערכים, בזה אחר זה. לכל 1.
אחד
מהם:
אם הוא לא אפשרי, המשך לפתרון הבא. אם הוא אפשרי, 1.

בחר משתנה אחר והפעל את הפונקציה

הרקורסיבית.

false, אם חזר פתרון, החזר אותו. אם חזר 2.
המשך
לפתרון הבא.
false. החזר 2.

- return failure



דגנית מוכתר-אבל

Backtracking Search

- שלב התחלתי אין הקצאות ריקות (משמאל למעלה).
- שלב A2 ומקצים לו ערך כלשהו, יום : בוחרים במשתנה שני (מימין למעלה).
- לאחר מכן, באמצעות הקצאה זו, אנו מפעילים את האלגוריתם שוב.
- כבר יש הקצאה, האלגוריתם ישקול את כעת, כאשר ל-A ויקצה לו את יום שני (משמאל למטה). B
- אז במקום להקצות ערך אם ההקצאה ל false מחזירה B בהינתן ההקצאה הקודמת, האלגוריתם ינסה להקצות ל C - יום שלישי (ימין למטה). ערך חדש ל B
- אם ההקצאה החדשה לA עונה על האילוצים (גם מול), B הוא ימשיך למשתנה חדש C
- תביא אם למשל, הקצאה של יום שלישי או רביעי ל B-A לכישלון, אז האלגוריתם יחזור בו ויחזור לשקול את ויקצה לו ערך אחר, יום שלישי.
- אם גם ביום שלישי ורביעי כשל חוזר, אז זה אומר שניסינו כל מטלה אפשרית והבעיה אינה ניתנת לפתרון.

Maintaining Arc-Consistency - היסק שילוב

- למרות שחיפוש נסוג יעיל יותר מחיפוש פשוט, הוא עדיין דורש כוח חישוב רב
  - אכיפת עקביות קשת, לעומת זאת, דורשת פחות משאבים
- על ידי שילוב של חיפוש במסלול אחורה עם הסקה (אכיפת עקביות קשת), נוכל להגיע לאלגוריתם יעיל יותר
- אלגוריתם זה יאכוף את Consistency-Arc Maintaining. אלגוריתם זה נקרא אלגוריתם עקביות הקשת לאחר כל הקצאה חדשה של חיפוש החזרה לאחור
  - ונאתחל תור של כל ACנקרא לאלגוריתם -3 X-לדוגמה, לאחר שנבצע הקצאה חדשה ל X (ולא תור של כל הקשתות בבעיה הקשתות של השכנים של)
- מתוקן ששומר על עקביות קשת, עם התוספות החדשות backtrack להלן אלגוריתם **מודגשות**

function Backtrack(*assignment, csp*):

- if *assignment* complete:
    - return *assignment*
- *var* = Select-Unassigned-Var(*assignment, csp*)
- for *value* in Domain-Values(*var, assignment, csp*):
    - if *value* consistent with *assignment*:
        - **add {*var* = *value*} to *assignment***
        - ***inferences* = Inference(*assignment, csp*)**
        - if *inferences* ≠ *failure*:
            - add *inferences* to *assignment*
        - *result* = Backtrack(*assignment, csp*)
        - if *result* ≠ *failure*:
            - return *result*
        - *remove* {*var* = *value*} **and *inferences*** from *assignment*
- return failure

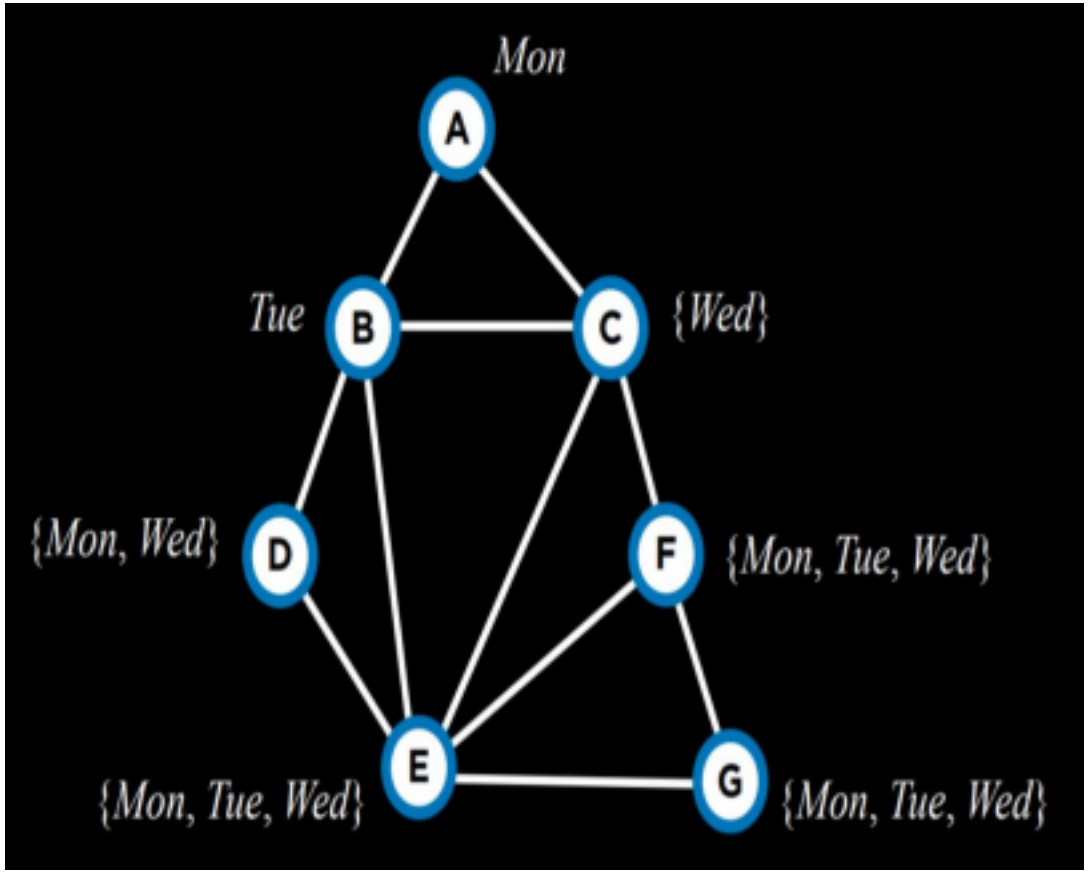
דגנית מוכתר-אבל

שילוב היסק - Maintaining Arc-Consistency

שילוב היוריסטיקות

- עד כה בחרנו השמה של ערך למשתנה באופן אקראי
- כדי לעשות את האלגוריתם יעיל יותר אפשר לשקול היוריסטיקה (כלל אצבע) שתביא לפתרון מהיר יותר

שילוב היוריסטיקות
MRV - Minimum Remaining

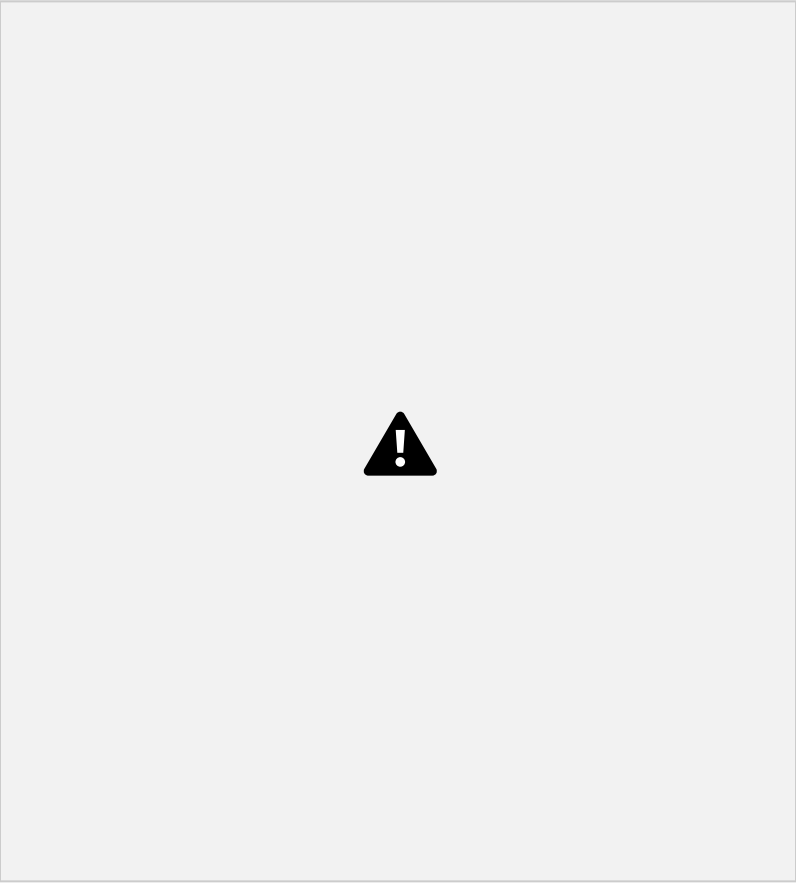


- מראש לבחון את רשימת הערכים **Values** ולנסות

לצמצם אותם ע"פ תנאים שמתאימים לבעיה

- זה יקטין את החיפוש ואם אי אפשר לפתור את הבעיה, נגלה את זה מהר יותר

בדוגמה מראש נסנן את הערכים של A ו B



דגנית מוכתר-אבל

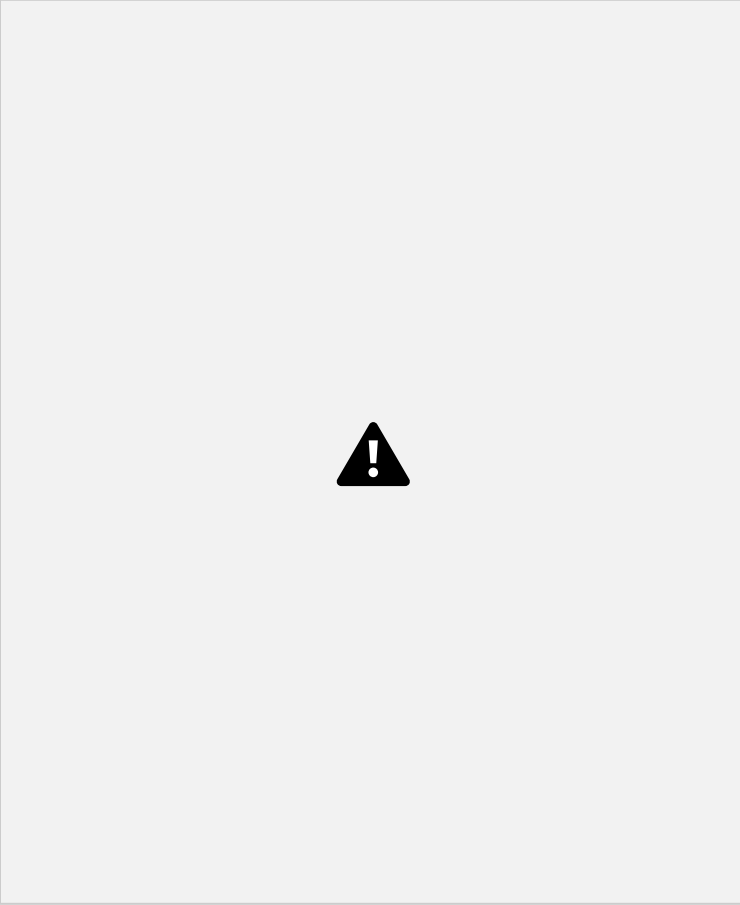
שילוב היוריסטיקות

Degree heuristic

● The **Degree** heuristic מתייחס לדרגה של צומת כלומר כמה משתנים/צמתים מקושרים לצומת מסוימת

● בחירת המשתנה עם הדרגה הגבוהה בהתחלה. עם הקצאה אחת, אנו מגבילים מספר משתנים אחרים, ומאיצים את תהליך האלגוריתם.

● בדוגמה לכל המשתנים יש אותו גודל דומיין ● אבל הדרגה של E (5) היא עם הדרגה הכי גבוהה ולכן נתחיל את האלגוריתם ממנה



דגנית מוכתר-אבל

שילוב היוריסטיקות

Least Constraining Values heuristic

- בוחרים ערך מתחום שיגביל את המשתנים האחרים הכי פחות.
- בעוד שבהיוריסטיקה של דרגה רצינו להשתמש במשתנה שסביר יותר להגביל משתנים אחרים, כאן אנחנו רוצים
שהמשתנה הזה יציב הכי פחות אילוצים על משתנים
אחרים.
- כלומר, אנו רוצים לאתר את מה שיכול להיות מקור
הצרות הפוטנציאלי הגדול ביותר (המשתנה בעל הדרגה
הגבוהה ביותר), ולאחר מכן להפוך אותו לפחות בעייתי
שנוכל (להקצות לו את הערך הכי פחות מגביל).
- זה Tue את הערך C בדוגמה אם נבחר עבור צומת
B, D, E ישפיע על האילוצים עם
- הוא יגרום פחות לאילוצים עם צמתים אחרים כי הוא משפיע רק על Wed אבל אם נבחר את הערך
B הצומת

אלגוריתמים נוספים מתורת הגרפים

- Hill climbimg

- Beam Search

האלגוריתם ההונגרי

דגנית מוכתר-אבל

Algorithm Hungarian - האלגוריתם ההונגרי

- האלגוריתם ההונגרי הוא אלגוריתם אופטימיזציה קומבינטורי
- גישה מהירה לפתרון בעיות השמה/אופטימזיציה בזמן פולינומי.
- דוגמה: באתר בנייה יש 4 מנופים. יש להקצות כל מנוף לעבודה אחת. הזמן הנדרש לכל עבודה עבור כל מנוף מוצג בטבלה למטה. מצא את ההקצאה הטובה ביותר של מנופים לעבודות כך שהזמן הדרוש לסיום העבודות יהיה מינימלי.
- הפתרון מימין

The Hungarian Algorithm

1. Find the minimum element from each row and substract that value from all the elements of the row.
2. Find the minimum element from each column and substract that value from all the elements of the column.
3. Let m=minimum number of lines required to cover all the zeroes in the table 4.

while(m!=number of row/columns)

   a. Find the minimum element from the uncovered elements
   b. Substract this element from all the other uncovered elements
   c. Add this element to the elements where the lines are intersecting
   d. Find new m

5. Use the zeroes to assign possible combinations- i.e wherever there is a zero present, task can be

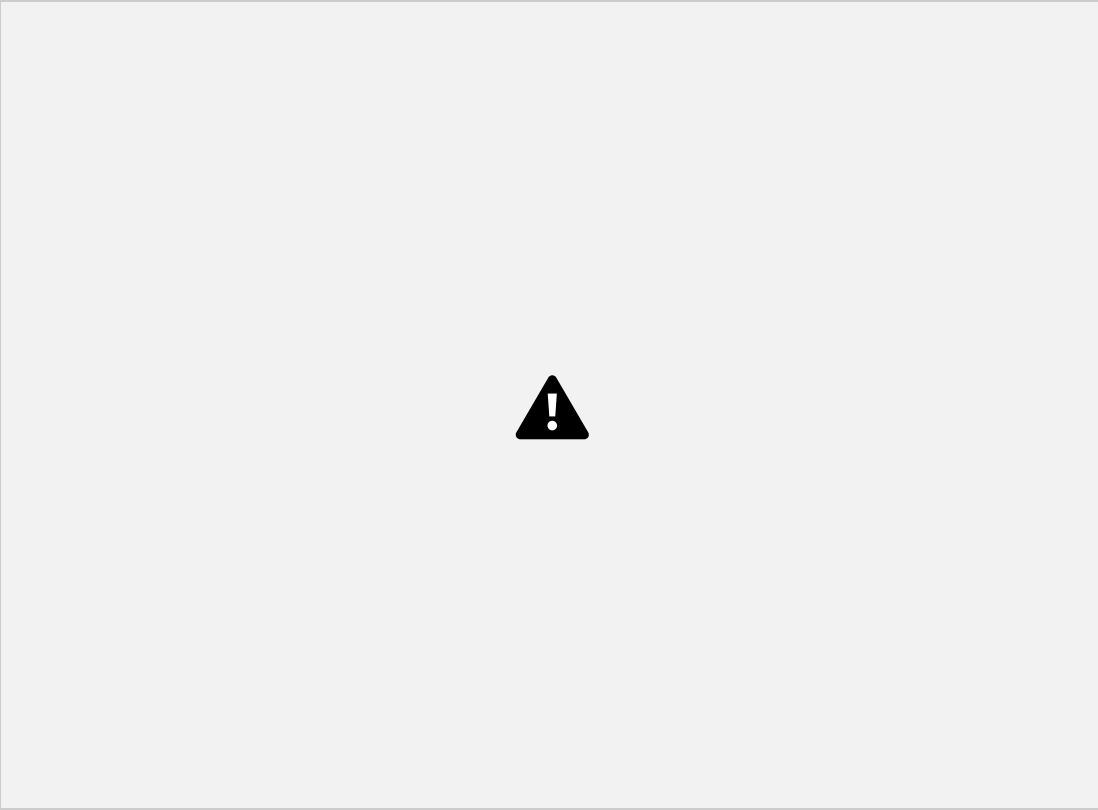
assigned.
6. Find the minimum cost
7. End

דוגמה והסבר

האלגוריתם ההונגרי

- סיבוכיות זמן
- מקטין סיבוכיות the algorithm Volgenant–Jonker שיפורים לאלגוריתם הונגרי
- (● טבלה דו ממדית ריבועית (מס עמודות = למספר שורות
- אם מספר העמודות לא שווה למספר השורות, צריך להוסיף עמודות/שורות דמה וזה מגדיל זמן פעולה
- הערך בתוך תא יכול לפעמים להיות מורכב מכמה גורמים ואז צריך לשקלל את הגורמים בצורה טובה
  - למשל בדוגמה הערך הוא זמן פעולה על משימה, אבל בשיבוץ ילד לבית ספר יש הרבה

שיקולים ולכן צריך לדעת להרכיב אותם לערך יחיד



דגנית מוכתר-אבל

bipartite graph

- גרף דו-צדדי הוא מקרה מיוחד בתורת

הגרפים בו כל משתנה נמצא באחד משתי קבוצות נפרדות,

- קשת מהווה התאמה בין צומת מקבוצה אחת לשניה שמקיימת אילוצים
- דוגמה: M-N משרות. לכל מועמדים לעבודה ו מועמד יש תת-קבוצה של משרות שהוא/היא מעוניין בהן. כל משרה נפתחת יכולה לקבל רק מועמד אחד ומועמד לעבודה יכול להתמנות למשרה אחת בלבד. מצא שיוך של משרות למועמדים כך שכמה שיותר מועמדים יגיעו לעבודה"

דגנית מוכתר-אבל

bipartite graph

- בעיית **MBP**

להיפתר ע"י המרה של המידע לרשת Maximum Bipartite Matching - יכולה

- אלגוריתמים להתאמה מקסימלית:
  - Ford-Fulkerson algorithm
  - Hopcroft-Karp Algorithm
- wiki/Bipartite_graph
- A NEW APPROACH TO THE MAXIMUM FLOW PROBLEM
- Appointment Scheduling Algorithm

בעיות תזמון

- Given N events with their starting and ending times, find a schedule that includes as many events as possible. It is not possible to select an event partially
- Scheduling in Greedy Algorithms